

Figure 2.7 Quantile scales take a range of values and reassign them into a set of equally sized bins.

Notice that the range values in figure 2.7 are fixed, and can accept text that may correspond to a particular CSS class, color, or other arbitrary value.

```
var qScaleName =
d3.scale.quantile().domain(sampleArray).range(["small","medium","large"]);
qScaleName (68);
qScaleName (20);
qScaleName (10000);
```

Returns "medium"

Returns "small"

Returns "large"

NESTING

Hierarchical representations of data are useful, and aren't limited to data with more traditional or explicit hierarchies, such as a dataset of parents and their children. We'll get into hierarchical data and representation in more detail in chapters 4 and 5, but in this chapter we'll use the D3 nesting function, which you can probably guess is called `d3.nest()`.

The concept behind nesting is that shared attributes of data can be used to sort them into discrete categories and subcategories. For instance, if we want to group tweets by the user who made them, then we'd use nesting:

```
d3.json("tweets.json",function(data) {
  var tweetData = data.tweets;
  var nestedTweets = d3.nest()
    .key(function(el) {return el.user})
    .entries(tweetData);
});
```

This nesting function combines the tweets into arrays under new objects labeled by the unique `user` attribute values, as shown in figure 2.8.

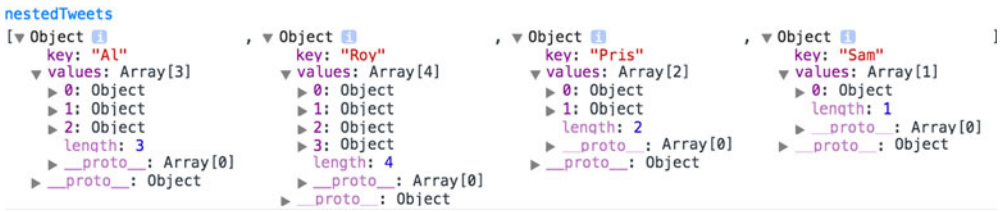


Figure 2.8 Objects nested into a new array are now child elements of a values array of newly created objects that have a key attribute set to the value used in the `d3.nest.key` function.